

[100 Puan] gem5 Simülatörü GShare ve GSelect Dallanma Öngörücüsü

gem5, C++ ve Python dillerinde yazılmış, birçok buyruk kümesi mimarisine (x86, ARM, MIPS, RISC-V vb.) dayalı işlemciyi simüle edebileceğiniz bir mimari simülatörü kütüphanesidir. (<https://www.gem5.org>) Siz de bu ödevde, gem5'e **GShare** ve **GSelect** Dallanma Öngörücülerini içeren özel bir öngörücü ekleyecek ve CoreMark kıyaslama programını (<https://github.com/eembc/coremark>) x86 mimarisinde farklı öngörücü parametreleriyle çalıştırarak sonuçları karşılaştıracaksınız. Öngörücüyü tasarlarken *Combining Branch Predictors* (<http://classweb.ece.umd.edu/enee646.F2007/combining.pdf>) adlı bildiriden yararlanabilirsiniz.

Şekil 1'de örnek bir GShare dallanma öngörücü yapısı verilmiştir. (GSelect öngörücünde, GShare öngörücünden farklı olarak XOR işlemi yerine bit birleştirme (concatenation) işlemi yapılır.)

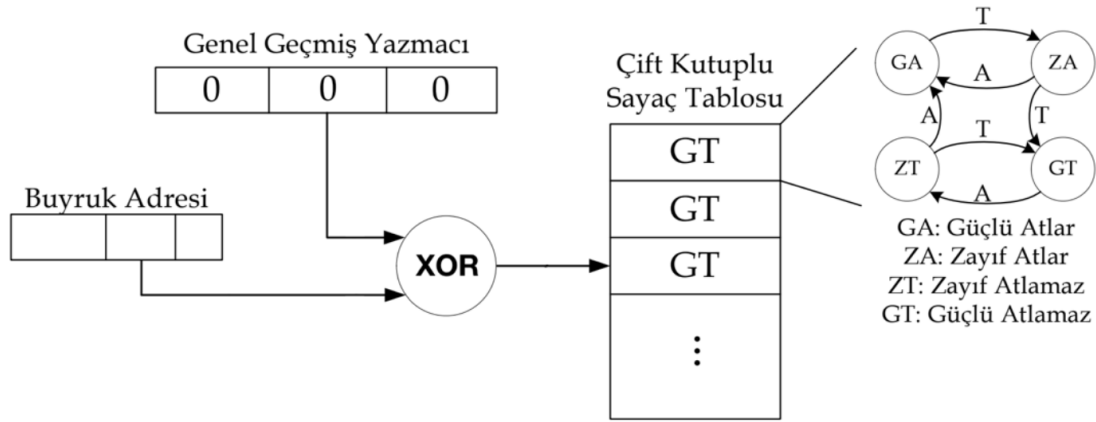


Figure 1: Örnek GShare Dallanma Öngörücü Yapısı

Tasarlamanız gereken özel dallanma öngörücü gereksinimleri şu şekildedir:

- Parametrik olarak GShare ya da GSelect öngörülerinden birini uygulayabilmelidir.
- Aşağıdaki parametrelere sahip olacaktır:
 - **tablo_boyutu (Param.Int):** Sayaç tablosunun satır sayısı.
Varsayılan değer: 1024
1024 ≤ alabileceği değerler ≤ 4096
 - **gecmis_uzunlugu (Param.Int):** Genel Geçmiş Yazmacının (GGY) bit cinsinden uzunluğu.
Varsayılan değer: 3
2 ≤ alabileceği değerler ≤ 30
 - **sayac_bit_sayisi (Param.Int):** Sayaç tablosundaki doyumluk sayaçlarının (saturating counters) bit sayısı.
Varsayılan değer: 2 (Çift Kutuplu Sayaç)
1 ≤ alabileceği değerler ≤ 8
 - **indeksleme_politikasi (Param.String):** Sayaç tablosuna erişirken kullanılacak indeksleme politikası. Modül aşağıdaki iki politikayı desteklemelidir (Varsayılan değer: GSHARE):
 - **GSHARE** (XOR tabanlı)
 - **GSELECT** (Birleştirme tabanlı)

- **baslangic_durumu (Param.String):** Dallanmanın başlangıç durumu. Modül aşağıdaki iki durumu desteklemelidir (Varsayılan değer: ATLAMAZ):
 - **ATLAR** (Dallanma atlar şeklinde başlatılır)
 - **ATLAMAZ** (Dallanma atlamaz şeklinde başlatılır)
- Simülasyon nesnesi aşağıdaki istatistikleri tutmalıdır:
 - **toplamDallanma:** Karşılaşılan toplam dallanma buyruğu sayısı
 - **basariliOngoru:** Başarılı tahmin edilen dallanma sayısı
 - **hataliOngoru:** Hatalı tahmin edilen dallanma sayısı
 - **ongoruBasarisiYuzdesi:** $100 \times \text{basariliOngoru} / \text{toplamDallanma}$
 - **guncellemeSayisi:** Öngörücü sayaç tablosunun güncellenme sayısı

gem5'e bir dallanma öngörücüsü eklemekten önce, gerekli bağımlılıkları yüklemeli ve kütüphaneyi doğru bir şekilde derlemelisiniz.

Not: Windows kullanıyorsanız Ubuntu WSL (<https://ubuntu.com/wsl>) ya da VirtualBox (<https://www.virtualbox.org>), macOS kullanıyorsanız yine VirtualBox, Parallels ya da Docker (<https://www.docker.com>) gibi bir program üzerinde x86 ubuntu sanal makine kullanmanız ve ubuntu için olan adımları takip etmeniz gerekmektedir.

Kütüphaneyi build etmek için gerekli araçları aşağıdaki gibi yükleyin:

```
sudo apt install build-essential
sudo apt install git
sudo apt install gcc g++
sudo apt install m4
sudo apt install scons
sudo apt install zlib1g
sudo apt install zlib1g-dev
sudo apt install python3-dev
```

gem5 kütüphanesini github'dan klonlayın ve d22064c1c05f5857431219da888de7e29a268c16 commit versiyonuna getirin (gem5 25.0.0.0 versiyonu) (<https://github.com/gem5/gem5/archive/refs/tags/v25.0.0.0.zip> adresinden de indirebilirsiniz):

```
git clone https://github.com/gem5/gem5
cd gem5
git checkout d22064c1c05f5857431219da888de7e29a268c16
python3 -m pip install -r requirements.txt
```

Kütüphaneyi elde ettikten sonra gem5 simülatörünü aşağıdaki gibi X86 buyruk kümesi mimarisi için build edin:

```
cd gem5
scons build/X86/gem5.opt -j1
```

Bu şekilde, gem5 simülatör programı (gem5.opt) *gem5/build/X86* dosya yolunda oluşmuş oldu. Burada daha hızlı build etmek için *-j1* ifadesinde "1" yerine daha fazla iş parçacığı (thread) atayabilirsiniz. Maksimum atayabileceğiniz iş parçacığı sayısını, terminalde *nproc* komutu ile öğrenebilirsiniz.

Bundan sonra kendi dallanma öngörücünüzü eklemek için **gem5/src/cpu/pred** dosya yoluna gitmeniz ve aşağıdaki dosyaları oluşturarak ya da değiştirerek **OzelOngorucu**'yü SimObject nesnesi olarak eklemeniz gerekmektedir:

- ozelongorucu.hh
- ozelongorucu.cc
- OzelOngorucu.py
- SConscript

Değişiklikler tamamlandıktan sonra, gem5 klasörüne gidin ve kütüphaneyi yeniden derleyin:

```
cd gem5
scons build/X86/gem5.opt -j1
```

Kütüphanenin sorunsuz bir şekilde yeniden build olduğundan emin olduktan sonra, dallanma öngörücü performansını ölçmek için simülatörde bir program çalıştırmanız gerekiyor. **CoreMark** benchmarkını klonlayın (<https://github.com/eembc/coremark/archive/refs/tags/v1.01.zip> adresinden de indirebilirsiniz) ve programı X86 Linux için derleyin:

```
git clone https://github.com/eembc/coremark
cd coremark
make compile PORT_DIR=linux
```

Eğer **CoreMark** başarıyla derlendiyse, **coremark** dizininin içinde **coremark.exe** dosyasını göreceksiniz.

Coremark programını özel dallanma öngörücünüz ile gem5 simülasyonunda çalıştırmak için bir simülasyon scripti oluşturmanız gerekmektedir. **simulasyon.py** adında bir python scripti oluşturun ve gem5 sistemini **O3CPU** ve **OzelOngorucu** nesneleri ile inşa edin. Belleğiniz **DDR4_2400_8x8** ve **1GB** boyutunda olsun. Simüle edilecek maksimum buyruk sayısını ise **10000000** (10 milyon) olarak belirleyin. Aşağıda gösterildiği gibi bu script üzerinden bayraklar (flags) yardımıyla özel öngörücü parametrelerini alabilmeli ve coremark'ı buna benzer spesifik yapılandırmalarla simüle edebilmelisiniz:

```
cd gem5
build/X86/gem5.opt simulasyon.py \
    --program="./coremark.exe" \
    --dosya_yolu="./cikti" \
    --tablo_boyutu=1024 \
    --gecmis_uzunlugu=3 \
    --sayac_bit_sayisi=2 \
    --indeksleme_politikasi="GSHARE" \
    --baslangic_durumu="ATLAMAZ"
```

Coremark programını GShare ve GSelect politikaları dahil olmak üzere farklı tablo boyutları, geçmiş uzunlukları, sayaç bit sayıları ve başlangıç durumları yapılandırmalarıyla simüle edin ve sonuçları maddeler halinde **<Soyad_Öğrenci Numarası>.pdf** dosyasına yazın. Programı simüle ettikten sonra, **cikti** dizininin içinde istatistikleri (özellikle **stats.txt** dosyasını) elde edeceksiniz. Bu istatistiklerden, farklı yapılandırmalar için öngörücünün nasıl davrandığından, coremark için en iyi sonuçları hangi yapılandırmada aldığınızdan ve her bir parametrenin öngörücü performansını (özellikle **ongoruBasarisiYuzdesi** değerini) nasıl etkilediğinden bahsedin.

GShare ve GSelect öngörücüleri arasında öngörü başarısı açısından belirgin bir fark olup olmadığını ve bunun mimari nedenini açıklayın. gem5'te varsayılan olarak bulunan LocalBP ve TournamentBP öngörücüleri ile tasarladığınız öngörücülerin başarımlarını karşılaştırın. Çok daha büyük tablo boyutları ve uzun geçmiş (GGY) değerleri kullanılarak %100 başarılı (0 hatalı) bir öngörünün mümkün olup olmadığını tartışın. Başka nesneler (örneğin L1 cache) eklemeyi deneyin (ancak gönderdiğiniz dosyada simülasyon kodunun varsayılan halini teslim edin) ve ön-bellek (cache) eklemenin ve boyutlarını değiştirmenin öngörü başarımını etkileyip etkilemediğini, etkiliyorsa nasıl etkilediğini analiz edin.

Öngörücüyü gem5'e eklemek için aşağıdaki ek kaynaklardan faydalanabilirsiniz:

Learning gem5 Tutorial

https://www.gem5.org/documentation/learning_gem5/introduction

Learning gem5 Tutorial: Creating a simple configuration script

https://www.gem5.org/documentation/learning_gem5/part1/simple_config

Learning gem5 Tutorial: Creating a very simple SimObject

https://www.gem5.org/documentation/learning_gem5/part2/helloobject

Learning gem5 Tutorial: Adding parameters to SimObjects and more events

https://www.gem5.org/documentation/learning_gem5/part2/parameters

Ödev Teslimi (Son Teslim Tarihi: 17.04.2026 23.59)

⚠ Çevrimiçi kaynakları veya üretken yapay zeka araçlarını kullanabilirsiniz, ancak tüm kodlarda intihal kontrolü yapılacak ve eşleşmeler -yapay zeka ile yapıp yapılmamasından bağımsız olarak- ödevden 0 puan alacaktır. Eğer kendiniz yaparsanız -yapay zekanın yardımıyla bile olsa- endişelenmenize gerek yok.

1-) ozelongorucu.hh

2-) ozelongorucu.cc

3-) OzelOngorucu.py

4-) SConscript

5-) simulasyon.py

6-) <Soyad_Öğrenci Numarası>.pdf

dosyalarını sıkıştırın ve <Soyad_Öğrenci Numarası>.zip olarak kaydederek <https://uzak.etu.edu.tr>'ye yükleyin.